

Module 06 Lab - Regression and Classification

ModelsObjective: To understand the difference between

- ✓ regression and classification and to build your first linear models for both tasks.**In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. ClassificationThis is the most fundamental distinction between types of supervised learning problems.*

Regression: The goal is to predict a **continuous numerical value**. *

Examples: Predicting the price of a house, the temperature tomorrow,

or the stock price.* **Classification:** The goal is to predict a **discrete category or class label**. * *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.**In this lab, we will tackle one of each.**

Part 2: Linear Regression**Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line

- ✓ through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".* **Problem:** We will predict the **Fare** of a Titanic passenger based on their **Age** and **Pclass**.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.')

# For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)

# Define features and target
features = ['Age', 'Pclass']
```

```
target_reg = 'Fare'

X_reg = df[features]
y_reg = df[target_reg]

# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=
```

- Task 1: Train and Evaluate a Linear Regression Model**Your Task:**1. Create an instance of the `LinearRegression` model.2. Train the model using the training data (`X_train_reg`, `y_train_reg`).3. Make predictions on the test data.4. Evaluate the model using `mean_squared_error`. This metric tells us the average of the squared differences between the predicted and actual values.

```
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
lr_model = LinearRegression()

# 2. Train the model
lr_model.fit(X_train_reg, y_train_reg)
print(lr_model.coef_)

# 3. Make predictions
y_pred_reg = lr_model.predict(X_test_reg)

# 4. Evaluate the model
mse = mean_squared_error(y_test_reg, y_pred_reg)
print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"The square root of this is {np.sqrt(mse):.2f}, meaning our model is off by about $58

[ -0.43695889 -38.29728935]
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01, meaning our model is off by about $48 on average.
```

- Part 3: Logistic Regression**Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Prepare data for classification
# We need to encode 'Sex' column
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
```

```
# Define features and target
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'

X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the data
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y_cls, test_size=0.
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the `LogisticRegression` model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using `accuracy_score`.

```
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
log_model = LogisticRegression()

# 2. Train the model
log_model.fit(X_train_cls, y_train_cls)

# 3. Make predictions
y_pred_cls = log_model.predict(X_test_cls)

# 4. Evaluate the model
accuracy = accuracy_score(y_test_cls, y_pred_cls)
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

Accuracy for Survival Prediction: 74.83%



Instructions: Answer the following questions in this markdown cell.1. **In your own words, what is the key difference between a regression problem and a classification problem?** The key difference between a regression and a classification problem is that a regression problem takes different data points to give you an estimate about what will happen with different data points. While classification also takes different inputs to predict and put them

in different categories instead of a numerical value changing based on inputs it goes in a bigger umbrella.

2. The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent? These numbers represent a couple of things: how big the number is. This shows how strong the relationship is, and positive means a higher accuracy while negative means lower accuracy

3. Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task? We don't use `accuracy_score` because it will use true and false, which wouldn't be very useful for predicting a number. We use MSE instead because since we're squaring numbers; bigger penalties if the model gets it wrong. Accuracy wouldn't be a good metric because fare is a numeric value while accuracy would be better used for putting things in groups

Module 06 Lab - Regression and Classification Models **Objective:** To understand the difference between regression and classification and to build your first linear models for both tasks. **In this lab, you will write the code to train and evaluate the models.**

Part 1: Regression vs. Classification This is the most fundamental distinction between types of supervised learning problems. *** Regression:** The goal is to predict a **continuous numerical value**. *** Examples:** Predicting the price of a house, the temperature tomorrow, or the stock price. *** Classification:** The goal is to predict a **discrete category or class label**. *** Examples:** Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not. **In this lab, we will tackle one of each.**

Part 2: Linear Regression **Concept:** Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept". *** Problem:** We will predict the **Fare** of a Titanic passenger based on their **Age** and **Pclass**.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load and prepare the data
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# For simplicity, we'll drop rows with missing age
df.dropna(subset=['Age'], inplace=True)

# Define features and target
features = ['Age', 'Pclass']
target_reg = 'Fare'
X_reg = df[features]
y_reg = df[target_reg]

# Split the data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)
```

Task 1: Train and Evaluate a Linear Regression Model ****Your Task:**** 1. Create an instance of the **LinearRegression** model. 2. Train the model using the training data (**X_train_reg**, **y_train_reg**). 3. Make predictions on the test data. 4. Evaluate the model using **mean_squared_error**. This metric tells us the average of the squared differences between the predicted and actual values.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Create the Linear Regression model.
lr_model = LinearRegression()

# Train the model so it can learn the relationship between age, class, and fare.
lr_model.fit(X_train_reg, y_train_reg);

# Predict fares for the passengers in the test set.
y_pred_reg = lr_model.predict(X_test_reg)
```

```
# Evaluate the size of the prediction errors.
mse = mean_squared_error(y_test_reg, y_pred_reg)
rmse = np.sqrt(mse)

print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"Root Mean Squared Error: ${rmse:.2f}")
print(f"Model coefficients [Age, Pclass]: {lr_model.coef_}")
print(f"Model intercept: {lr_model.intercept_:.2f}")
```

```
Mean Squared Error for Fare Prediction: 3364.92
Root Mean Squared Error: $58.01
Model coefficients [Age, Pclass]: [ -0.43695889 -38.29728935]
Model intercept: 133.31
```

Part 3: Logistic Regression **Concept:** Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.* **Problem:** We will predict whether a passenger **Survived** based on their **Age**, **Pclass**, and **Sex**.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Convert the text categories in Sex into numeric values for the model.
# male = 0 and female = 1
df["Sex"] = df["Sex"].map({"male": 0, "female": 1})

# Select the input features and categorical target for classification.
features_cls = ["Age", "Pclass", "Sex"]
target_cls = "Survived"

X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the classification data into training and testing sets.
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls,
    y_cls,
    test_size=0.20,
    random_state=42
)

print(f"Classification training rows: {len(X_train_cls)}")
print(f"Classification testing rows: {len(X_test_cls)}")
```

```
Classification training rows: 571
Classification testing rows: 143
```

Task 2: Train and Evaluate a Logistic Regression Model**Your Task:**1. Create an instance of the **LogisticRegression** model.2. Train the model using the classification training data.3. Make predictions on the test data.4. Evaluate the model using **accuracy_score**.

```
# Create the Logistic Regression classifier.
# A larger max_iter value gives the training process enough iterations to converge.
log_model = LogisticRegression(max_iter=1000)

# Train the classifier using the passenger characteristics and survival labels.
log_model.fit(X_train_cls, y_train_cls)

# Predict whether each passenger in the test set survived.
y_pred_cls = log_model.predict(X_test_cls)

# Calculate the percentage of test predictions that were correct.
accuracy = accuracy_score(y_test_cls, y_pred_cls)

print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

 Knowledge CheckInstructions: Answer the following questions in this markdown cell.

1. **In your own words, what is the key difference between a regression problem and a classification problem?** A regression problem predicts a continuous numerical value, while a classification problem predicts a category or class. In this lab, predicting a passenger's fare is regression because the answer can be many different dollar amounts. Predicting survival is classification because the result is one of two labels: survived or did not survive.
2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?** The coefficients show how much the model's predicted fare changes when one feature increases by one unit while the other feature stays the same. The Age coefficient is approximately 0.44, which means the predicted fare decreases by about \$0.44 for each additional year of age when passenger class is held constant. The Pclass coefficient is approximately -38.30, which means moving from one class number to the next, such as first class to second class, lowers the predicted fare by about 38.30 when age is held constant. These values describe the model's learned relationships and do not prove that either feature directly causes a fare change.
3. **Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model?** Mean squared error is appropriate for regression because it measures the numerical distance between each predicted fare and the actual fare. Squaring the errors also gives larger mistakes more weight. Accuracy is appropriate for classification because it measures the percentage of correctly predicted class labels. Accuracy would not work well for fare prediction because fares are continuous values, and a prediction such as 49.50 would normally not exactly match an actual fare of \$50.00 even though the prediction is very close.


```

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Titanic dataset
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')

# drop rows with missing Age values
df.dropna(subset=['Age'], inplace=True)

# Define features and target for regression
features = ['Age', 'Pclass']
target_reg = 'Fare'

X_reg = df[features]
y_reg = df[target_reg]

# training and testing sets data split
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg,
    y_reg,
    test_size=0.2,
    random_state=42
)

print("Regression training data shape:", X_train_reg.shape)
print("Regression testing data shape:", X_test_reg.shape)
print("\nFirst five rows of regression features:")
print(X_reg.head())

```

Regression training data shape: (571, 2)
 Regression testing data shape: (143, 2)

First five rows of regression features:

	Age	Pclass
0	22.0	3
1	38.0	1
2	26.0	3
3	35.0	1
4	35.0	3

```

# Task 1

# model instance
lr_model = LinearRegression()

# training the model
lr_model.fit(X_train_reg, y_train_reg)

# predictions
y_pred_reg = lr_model.predict(X_test_reg)

# evaluation
mse = mean_squared_error(y_test_reg, y_pred_reg)
rmse = np.sqrt(mse)

print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")
print(f"Root Mean Squared Error: {rmse:.2f}")

print("\nLinear Regression Coefficients:")
for feature, coefficient in zip(features, lr_model.coef_):
    print(f"{feature}: {coefficient:.2f}")

print(f"\nIntercept: {lr_model.intercept_:.2f}")

print("\nSample predictions:")
comparison_df = pd.DataFrame({
    'Actual Fare': y_test_reg.values,
    'Predicted Fare': y_pred_reg
})

print(comparison_df.head(10))

```

Mean Squared Error for Fare Prediction: 3364.92
 Root Mean Squared Error: 58.01

Linear Regression Coefficients:

Age: -0.44
 Pclass: -38.30

Intercept: 133.31

Sample predictions:

	Actual Fare	Predicted Fare
0	13.0000	38.360415
1	18.7500	55.401811
2	26.0000	44.040880
3	69.3000	84.522964
4	6.4500	-0.373833
5	36.7500	53.217017
6	15.8500	3.995755
7	23.0000	33.116908
8	7.8958	6.180550
9	7.9250	8.365344

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Prepare data for classification
# Logistic Regression predicts a category, so here we are predicting Survived.

# encode the Sex column. models need numbers
df['Sex'] = df['Sex'].map({
    'male': 0,
    'female': 1
})

# Define features and target for classification
features_cls = ['Age', 'Pclass', 'Sex']
target_cls = 'Survived'

X_cls = df[features_cls]
y_cls = df[target_cls]

# Split the data into training and testing sets
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(
    X_cls,
    y_cls,
    test_size=0.2,
    random_state=42
)

print("Classification training data shape:", X_train_cls.shape)
print("Classification testing data shape:", X_test_cls.shape)
print("\nFirst five rows of classification features:")
print(X_cls.head())
```

Classification training data shape: (571, 3)
 Classification testing data shape: (143, 3)

First five rows of classification features:

	Age	Pclass	Sex
0	22.0	3	0
1	38.0	1	1
2	26.0	3	1
3	35.0	1	1
4	35.0	3	0

Task 2

```
# model instance
log_model = LogisticRegression(random_state=42)

# train the model
log_model.fit(X_train_cls, y_train_cls)

# predictions
y_pred_cls = log_model.predict(X_test_cls)

# evaluation
accuracy = accuracy_score(y_test_cls, y_pred_cls)
```

```

print(f"Accuracy for Survival Prediction: {accuracy:.2%}")

print("\nClassification Report:")
print(classification_report(y_test_cls, y_pred_cls))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test_cls, y_pred_cls))

print("\nLogistic Regression Coefficients:")
for feature, coefficient in zip(features_cls, log_model.coef_[0]):
    print(f"{feature}: {coefficient:.2f}")

print(f"\nIntercept: {log_model.intercept_[0]:.2f}")

```

Accuracy for Survival Prediction: 74.83%

Classification Report:

	precision	recall	f1-score	support
0	0.80	0.78	0.79	87
1	0.67	0.70	0.68	56
accuracy			0.75	143
macro avg	0.74	0.74	0.74	143
weighted avg	0.75	0.75	0.75	143

Confusion Matrix:

```
[[68 19]
 [17 39]]
```

Logistic Regression Coefficients:

Age: -0.04
Pclass: -1.25
Sex: 2.53

Intercept: 2.64

Knowledge Check

- In your own words, what is the key difference between a regression problem and a classification problem?
 - The main difference is that regression predicts a number, while classification predicts a category. In this lab, predicting Fare is regression because fare is a continuous dollar amount, but predicting Survived is classification because the answer is either 0 or 1.
- The LinearRegression model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?
 - The `.coef_` values show how much each feature affects the model's prediction. For the Linear Regression model, the coefficients explain how Age and Pclass are connected to the predicted fare. For example, if a coefficient is positive, the predicted fare goes up as that feature increases. If it is negative, the predicted fare goes down as that feature increases.
- Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model? Why wouldn't accuracy be a good metric for the fare prediction task?
 - We used `mean_squared_error` for the regression model because fare prediction is a number-based problem, so we need to measure how far off the predicted fare was from the actual fare. We used `accuracy_score` for the classification model because survival prediction is a category problem, so we can count how many predictions were correct. Accuracy would not be good for fare prediction because fare is not simply right or wrong like a class label; a prediction can be close, far off, or somewhere in between.

